# PROYECTO INTEGRADOR 2 - SISTEMA DE GESTIÓN DE ASIENTOS DE CINE




## FACULTAD DE CIENCIAS Y TECNOLOGÍAS DE LA INFORMACIÓN

**MATERIA: ESTRUCTURA DE DATOS**

**INTEGRANTES:**
**VALERIA BERENICE ALVARADO CASTILLO**
**ROSALIA CHAVEZ LAGUNAS**

**MAESTRO: ROBERTO ROSARIO**


**FECHA: 8 DE NOVIEMBRE 2026**

## 1. INTRODUCCIÓN

El presente proyecto integrador aborda el desarrollo e implementación de un sistema de consola en lenguaje C# diseñado para digitalizar la administración y control de asientos dentro de una sala de cine. El objetivo primordial es erradicar errores comunes en las reservaciones manuales mediante el uso de estructuras de datos bidimensionales (matrices), permitiendo visualizar en tiempo real el estado de ocupación y disponibilidad de la sala.

## 2. PLANTEAMIENTO DEL PROBLEMA

Un cine requiere modernizar su proceso de venta y control de boletos. Actualmente, el registro manual de asientos genera duplicidad en las reservaciones y falta de certeza sobre la disponibilidad real de la sala. Se necesita un sistema automatizado que permita visualizar una matriz interactiva de 30 asientos (5 filas por 6 columnas), ofreciendo funciones para reservar, liberar y consultar disponibilidad de forma precisa y eficiente.

## 3. OBJETIVOS

3.1 Objetivo General: Desarrollar un sistema de consola modular en C# que organice y gestione el estado de 30 asientos en una matriz bidimensional, optimizando la asignación, liberación y consulta de espacios en tiempo real.

### 3.2 Objetivos Específicos:

1. Representar visualmente la sala de cine mediante una matriz de 5x6 utilizando notaciones claras de disponibilidad (**[ O ]** para Disponible y **[ X ]** para Ocupado).
2. Diseñar una arquitectura modular mantenible basada en métodos y estructuras de control claras (como **switch-case** y ciclos **for** anidados).
3. Garantizar un avance progresivo mediante control de versiones en GitHub con la participación equitativa de los dos integrantes del equipo.

## 4. DÍA 1: ANÁLISIS DEL PROBLEMA

### 4.1 Identificación de Componentes del Sistema

Entradas (Input):

- Selección de opciones dentro del menú principal (números del 1 al 5).
- Coordenadas numéricas de posición de asiento (Fila de 1 a 5, Columna de 1 a 6).

Procesos (Process):

- Inicialización automatizada de la matriz bidimensional **SalaDeCine[5, 6]** con valores iniciales de disponibilidad (**"[ O ]"**).

- Evaluación de la opción ingresada por el usuario mediante una estructura de control selectiva **switch-case.**
- Recorrido mediante ciclos anidados **for** para reflejar el estado actualizado de la matriz en la pantalla de la consola.

Salidas (Output):

- Menú interactivo de gestión en consola.
- Representación gráfica en consola de la pantalla del cine y la cuadrícula de asientos ordenada por filas y columnas.
- Mensajes claros de confirmación o alerta de error ante la navegación y reservación.

### 4.2 Justificación de la Estructura de Datos

Se eligió una matriz bidimensional (**string[,]**) debido a que la distribución física de una sala de cine responde directamente a un plano rectangular compuesto por filas y columnas. Esta estructura de datos permite un acceso directo indexado $O(1)$ a cualquier asiento utilizando simplemente sus coordenadas **[fila, columna]**.

## 5. DÍA 2: DISEÑO DE LA SOLUCIÓN

### 5.1 Algoritmo General (Pseudocódigo)

5.1 Algoritmo General (Pseudocódigo)

```
Algoritmo GestionSistemaCine
   Definir SalaDeCine como Matriz[5, 6] de Texto
   Definir opción como Texto
   Definir ejecutando como Lógico

   InicializarSala()
   ejecutando <- Verdadero

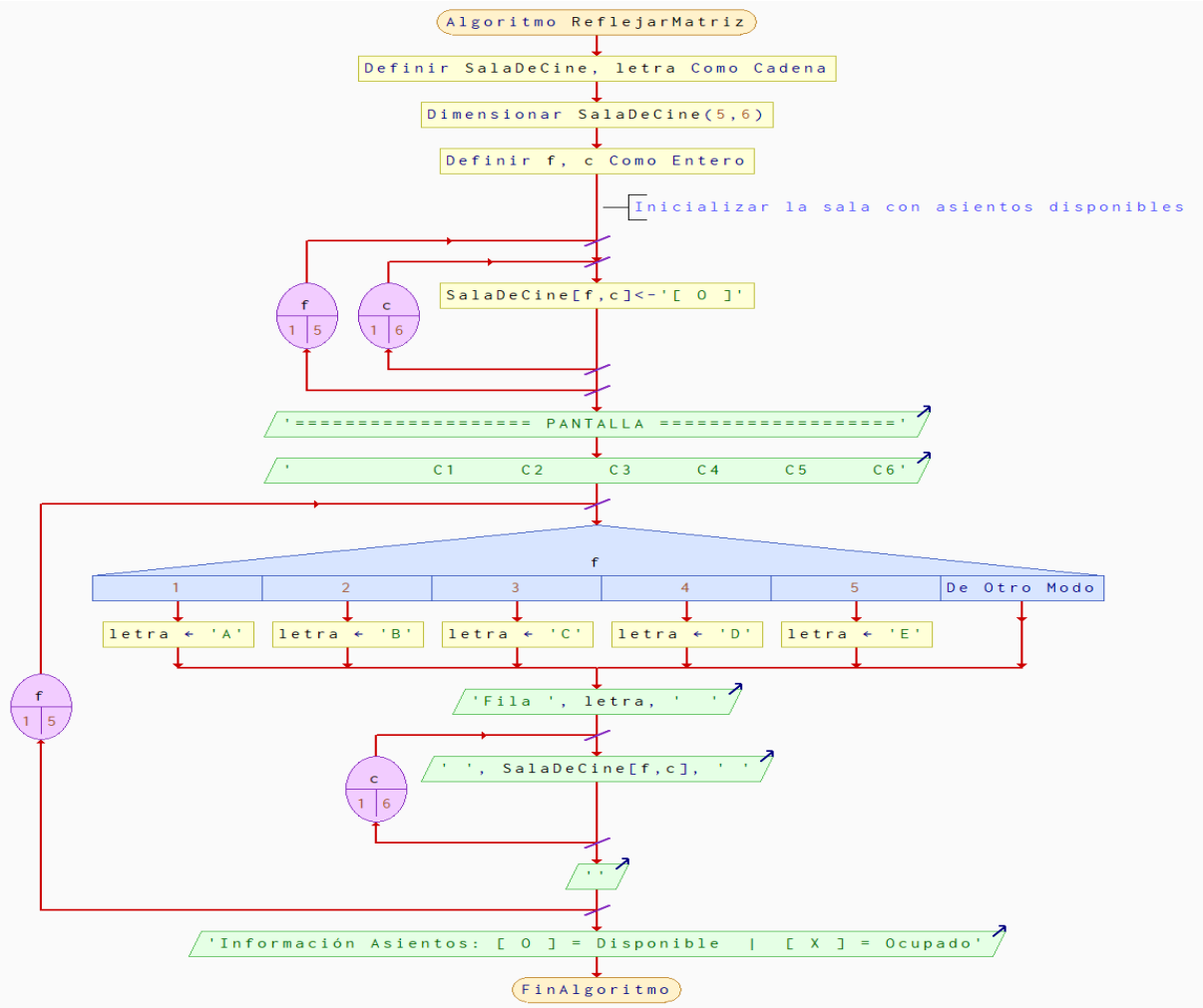
   Mientras ejecutando = Verdadero Hacer
      LimpiarPantalla()
      MostrarMenu()
      Leer opcion

      Según opcion Hacer
         Caso "1":
            ReflejarMatriz()
            Pausar()
         Caso "2":
            ReservarAsiento()
            Pausar()
```

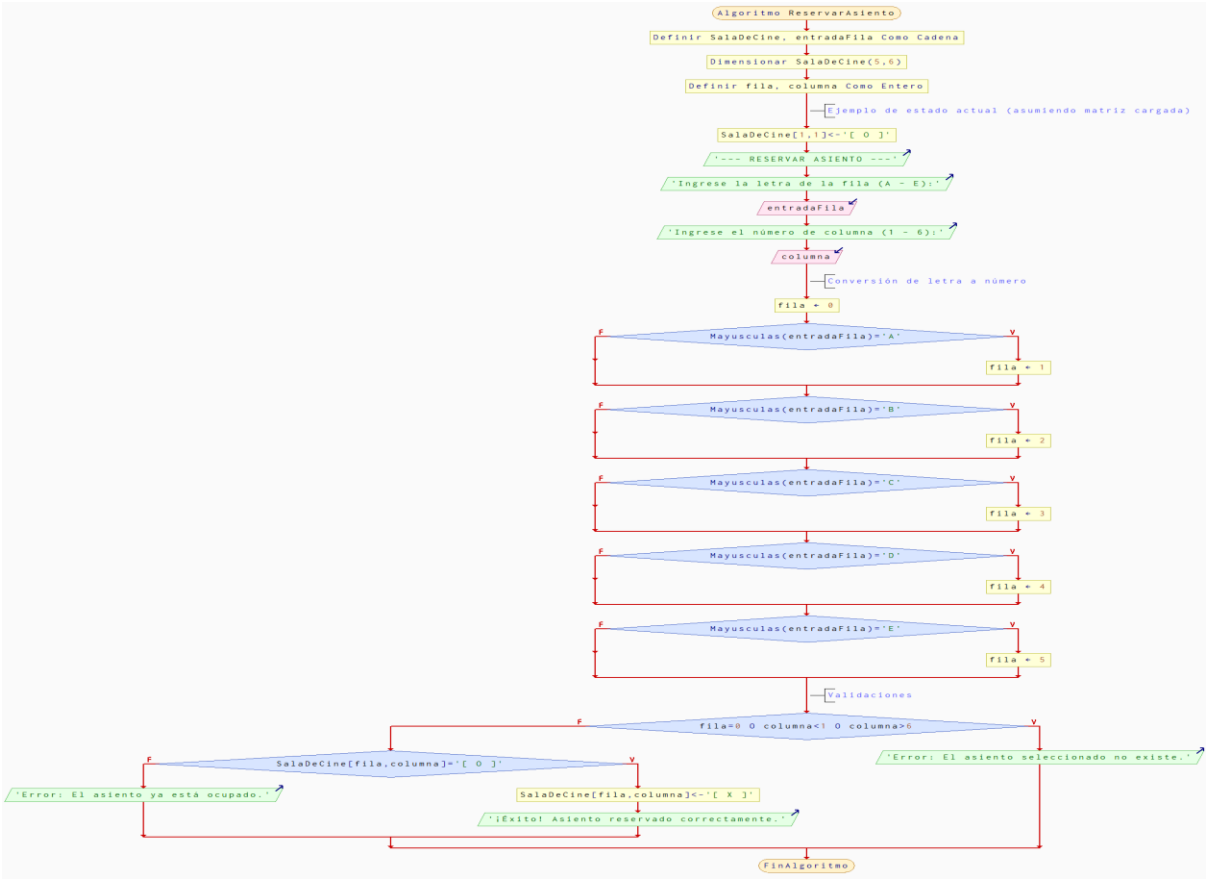
```
            Caso "3":
                LiberarAsiento()
                Pausar()
            Caso "4":
                ConsultarDisponibilidad()
                Pausar()
            Caso "5":
                ejecutando <- Falso
                Escribir "¡Saliendo del sistema!"
            De Otro Modo:
                Escribir "Opción no válida. Intente de nuevo."
                Pausar()
        FinSegún
    FinMientras
FinAlgoritmo
```

### 5.2 Diagramas de Flujo

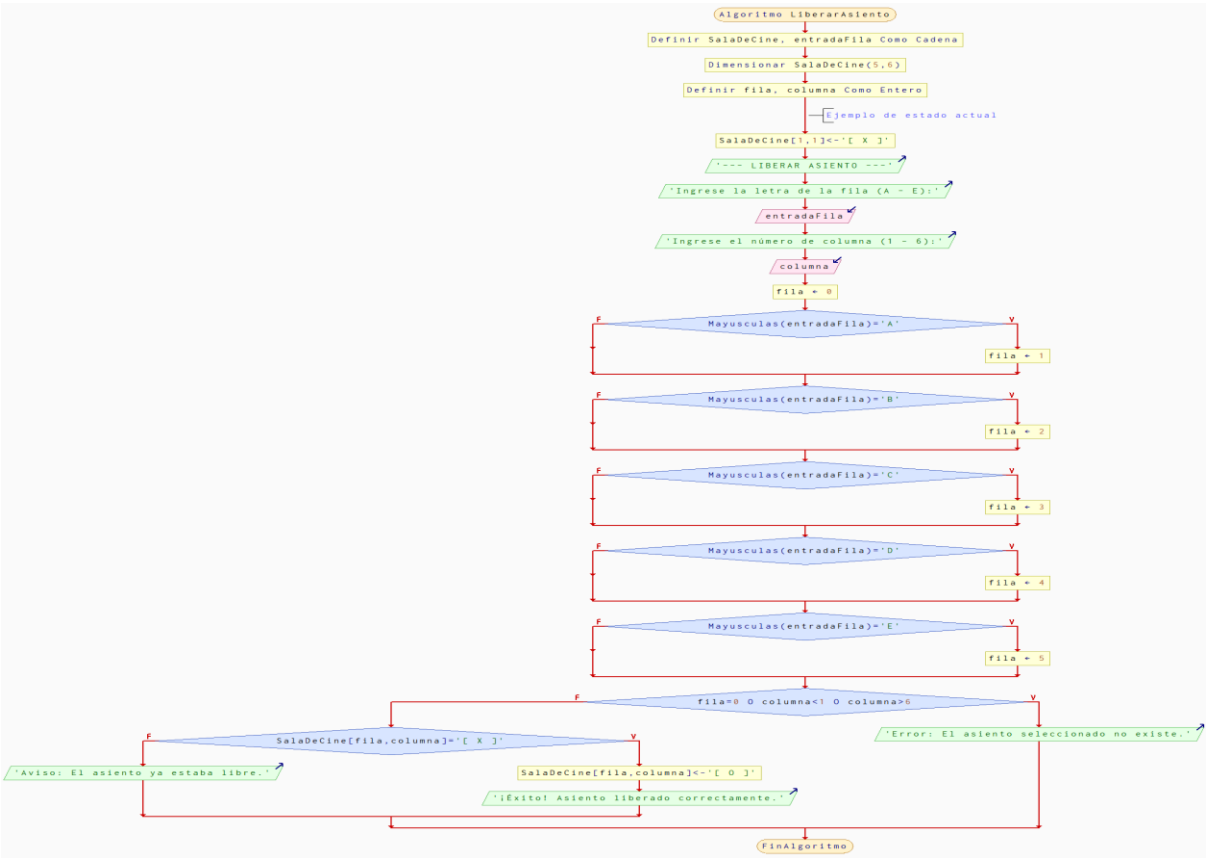
1. Inicializar y Mostrar Estado de la Sala (Notación Pantalla y Asientos)



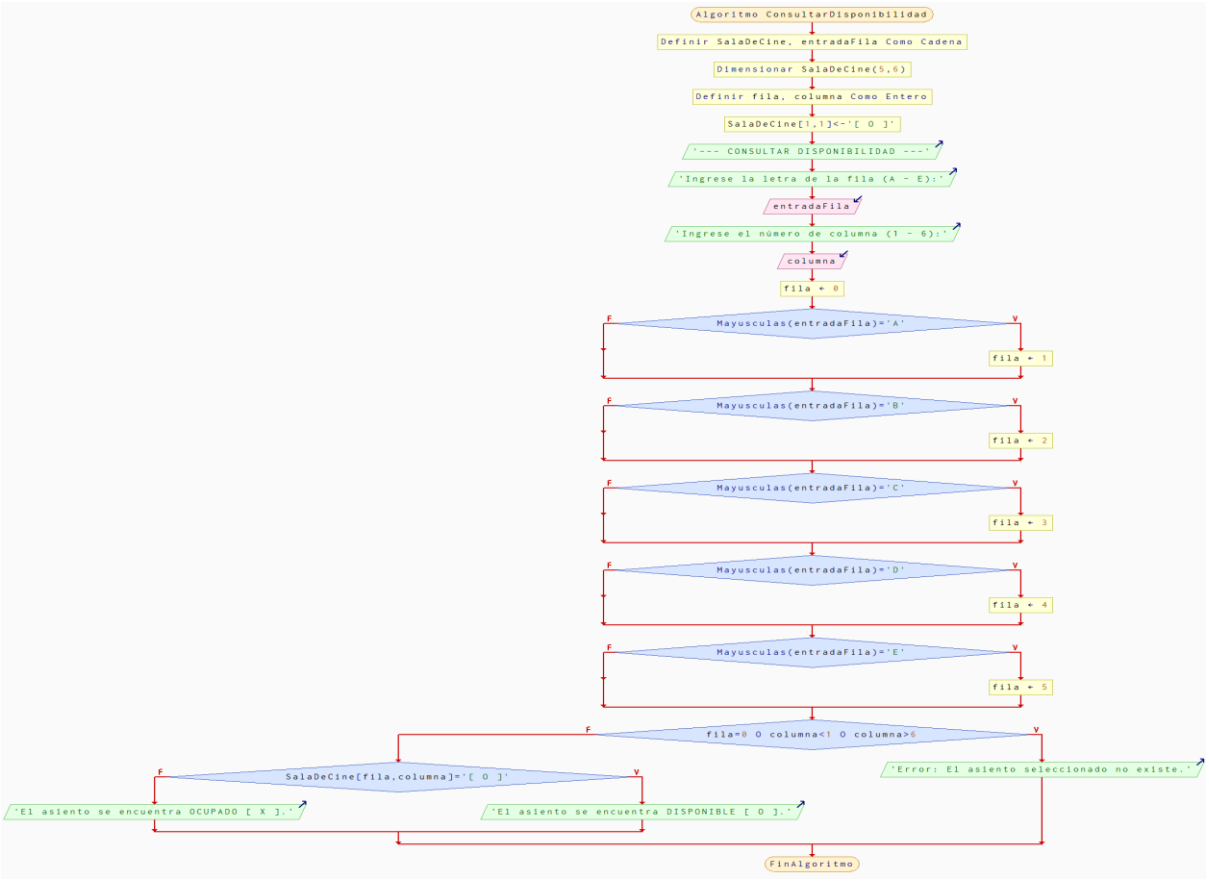
2. Reservar Asiento



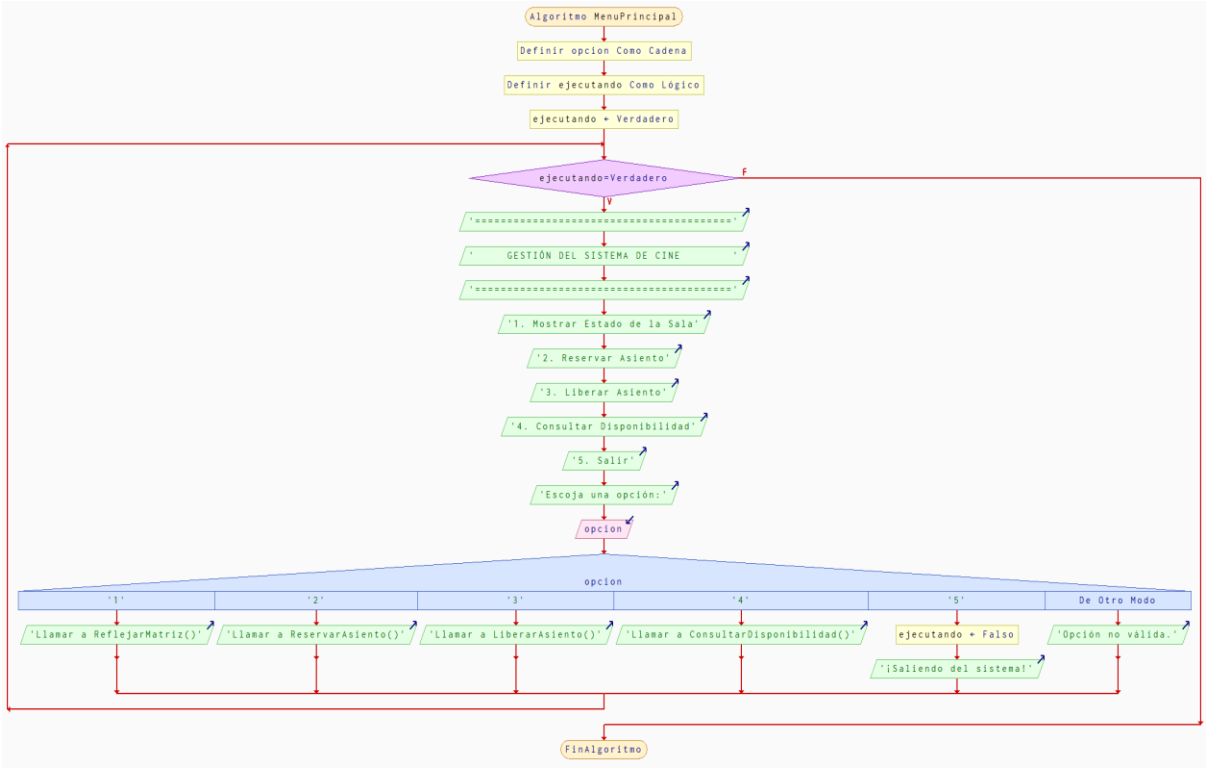

3. Liberar Asiento



4. Consultar Disponibilidad de Asiento




5. Estructura de Control del Menú Principal



## 6. DÍA 3: DISEÑO DE LA SOLUCIÓN

### 6.1 Definición de la Estructura de Datos

Para representar la sala de cine se utilizó una **matriz bidimensional estática** de tipo string con dimensiones de 5 filas por 6 columnas, proporcionando una capacidad total de 30 asientos.

- **Matriz SalaDeCine[5, 6]:** Almacena el estado visual de cada asiento mediante los identificadores "[ O ]" para asientos disponibles y "[ X ]" para asientos ocupados.
- **Arreglo de Mapeo LetrasFilas[5]:** Arreglo unidimensional de caracteres (['A', 'B', 'C', 'D', 'E']) que permite mapear los índices numéricos de las filas (0 a 4) a sus etiquetas alfabéticas correspondientes para la vista del usuario.

### 6.2 Arquitectura Modular de Funciones

El sistema se dividió en métodos auxiliares especializados para garantizar el principio de responsabilidad única y facilitar el mantenimiento del código:

1. **InicializarSala()**: Recorre la matriz mediante dos ciclos for anidados e inicializa todas las posiciones [f, c] con el valor predeterminado "[ O ]".
2. **ReflejarMatriz()**: Imprime el encabezado de la pantalla, las etiquetas de las columnas (C1 a C6) y recorre la matriz imprimiendo cada fila con su respectiva letra (A a E) y el estado de sus asientos.
3. **ReservarAsiento()**: Solicita la coordenada del asiento (letra de fila y número de columna), valida la existencia del asiento y cambia su estado a "[ X ]" si se encontraba disponible.
4. **LiberarAsiento()**: Recibe la coordenada del asiento, verifica si está ocupado ("[ X ]") y restablece su valor a "[ O ]".
5. **ConsultarDisponibilidad()**: Permite consultar el estado actual de un asiento específico sin modificar su valor en la matriz.
6. **ConvertirLetraAIndice(string letra)**: Método auxiliar que convierte la entrada alfabética del usuario (A-E) al índice numérico entero de la matriz (0-4), retornando -1 en caso de recibir una letra inválida.
7. **Pausar()**: Pausa la ejecución del programa hasta que el usuario presione una tecla, evitando que la consola se limpie automáticamente antes de leer la respuesta.